

Design Specification

Project: Eric Li — Infrastructure, Systems Engineering and Portfolio

Website: [ERI / CLI](#)

This document covers the portfolio's design system, page structure, interactive landscape and delivery architecture.

Specification baseline — Canonical target specification defining the visual design system, interaction models, and delivery architecture for the production repository.

Contents

1. [Project overview](#)
2. [Design direction and visual system](#)
3. [Pages, navigation and content placement](#)
4. [Interactive landscape](#)
5. [Responsive behavior and accessibility](#)
6. [Loading and rendering lifecycle](#)
7. [Editorial approach and publishing](#)
8. [Technical reference](#)
9. [Appendix: implementation excerpts](#)

1. Project overview

ERI / CLI presents Eric Li's work with self-hosted homelabs, Linux clusters, containers and storage, alongside his development toward network engineering and site reliability. The portfolio gives visitors a concise introduction, selected project evidence and a route into longer technical writing.

The homepage leads with operational context: 29 users, 23 active users and 5,769 hours logged in the preceding year across UniFi, Proxmox and Docker. These are presented as portfolio content, with supporting project explanations, rather than as a live monitoring dashboard.

The design combines large editorial typography with a dark background and a procedural wireframe landscape. The landscape gives the site a recognizable visual identity; the text explains what Eric operates, how the systems work and what he has learned from maintaining them.

The main priorities are to:

- Explain the work through named systems, project details and concrete figures.
- Give headlines and technical prose a clear reading order.
- Use the landscape as an authored visual element, without presenting decoration as live telemetry.
- Make room for unfinished notes, incident reviews and operational mistakes when they help explain the work.

2. Design direction and visual system

The visual direction is *The Magic 3*: tightly set serif headlines, an uneven three-column grid and a wireframe landscape whose individual pillars remain visible. The large type gives the opening statement room to speak; smaller labels keep operational figures and project facts easy to scan.

The `3 × 3 × 3 × 3` concept is a design constraint: three type families, three core colors, three homepage sections and three desktop columns. It provides consistency without requiring every component or explanation to repeat the same pattern.

Typography

Each family has a defined role:

FAMILY	USE	TREATMENT
PP Kyoto	Hero headline, major section headings and project titles	Display serif; line height <code>0.78</code> – <code>0.85</code> , tracking <code>-0.055em</code> – <code>-0.065em</code>
AUTHENTIC Sans	Body copy, project descriptions, article prose and primary navigation	The main reading face; registered as <code>'AUTHENTIC Sans'</code> in the CSS token
Pitch Sans	Metadata, timestamps, hardware statistics, categories and technical tags	Monospaced text, uppercase tracking where appropriate and <code>font-variant-numeric: tabular-nums</code>

PP Kyoto’s tight line spacing makes a multiline headline read as one compact shape. Longer explanations move into AUTHENTIC Sans. Pitch Sans handles details such as `2026-08-10 · 3 MIN READ`, giving dates and measurements a distinct place on the page.

Color

Black and white carry most of the interface. Blue marks selected interactions and higher terrain elevations.

COLOR	VALUE	USE
Void Black	<code>#08090C</code>	Main background, <code>--ink</code>
Dark background variant	<code>#0A0B0E</code>	Supporting background tone, <code>--structure</code>
Titan White	<code>#E5E7EB</code>	Text, structural lines and low terrain elevations
International Klein Blue	<code>#002FA7</code>	Accent token, terrain summits, active link underlines and cursor highlights

Titan White uses calibrated contrast levels to establish clear visual hierarchy:

- **100%:** primary headlines and active states (`--signal` , `#E5E7EB`).
- **Calibrated High (82%):** body prose and secondary descriptions (`--signal-high` , `rgba(229, 231, 235, .82)`), calibrated to exceed WCAG AAA contrast ratio on `--ink` .
- **48%:** metadata, hardware tags and captions (`--signal-48` , `#8892a0`).
- **Subtle (38%):** divider lines and subtle gridlines (`--signal-24` , `rgba(75, 85, 101, .38)`).

- **Faint (18%):** structural borders and inactive borders (`--signal-12` , `rgba(75, 85, 101, .18)`).

Blue is reserved for focal details rather than broad background fills. Active links use `text-decoration-color: var(--accent)`. Terrain brightness is handled separately in the elevation shader (Section 4).

Desktop layout

The desktop grid uses a nominal **23 : 49 : 28** column ratio. The wider center gives headlines and featured projects more room; the side columns hold statistics, context and supporting facts.

COLUMN	HOME PAGE INTRODUCTION	SELECTED PROJECTS
Left, 23 parts	Operational statistics	Project selector
Center, 49 parts	Main headline	Featured project card
Right, 28 parts	Background and explanation	Project facts

The grid uses fractional tracks with minimum widths of `190px` , `410px` and `240px` . The ratio therefore describes the intended distribution, not an exact percentage of the full viewport at every width. Gaps and minimum track sizes also affect the result. See Appendix A for the grid CSS.

3. Pages, navigation and content placement

The homepage introduces the work in three full-screen sections on desktop. Dedicated pages provide more detail.

Homepage

Introduction and operational evidence. The opening section places statistics on the left, the main statement in the center and context on the right. The headline reads, “I build, deploy and operate self-hosted infrastructure and software.” Supporting copy introduces the UniFi network, Proxmox cluster and related services. The landscape sits below and behind the content.

Selected projects. The second section presents three interactive production stacks. A selector, featured project card and facts panel follow the same desktop column structure. The landscape moves toward the top of the viewport as visitors enter this section.

About, writing and contact. The final section, `#continuation` , provides an About excerpt, a selected blog post and contact details. The About excerpt opens with “I started building PCs at 13.” and links to “About Eric.” The blog area combines a category label, date and reading-time metadata, a selected title and an “All blog posts” link. The landscape recedes to the upper right so the text becomes the main focus.

The three type families remain available across this section, but each element uses the role it needs. Contact details do not need a display headline simply to repeat the pattern elsewhere.

Dedicated pages

DESTINATION	PURPOSE
<code>/projects.html</code>	The infrastructure and project archive
<code>/about.html</code>	Personal background, narrative and roadmap
<code>/blog/</code>	Technical articles and incident reviews generated from the Markdown source collection

The header contains **Projects**, **About** and **Blog**. These links all lead to pages. Email is placed with contact information lower in the reading sequence, where visitors can choose it after reviewing the work. This also avoids making an email-client action part of the primary page navigation.

Contact

The homepage contact area displays `helloworld@n5hq.me` alongside profile links. GitHub, LinkedIn, and Blog (`blog/index.html`) are listed consistently.

Footers on `about.html`, `projects.html` and `blog/` follow natural document flow and contain email and social links, with contrasting hover treatments.

4. Interactive landscape

The “Faultline” landscape is a procedural Three.js WebGL scene. Low ripples, terraced peaks and an overhead lattice change the appearance of the pillar field as the reader moves through the homepage. Cursor movement and clicks bring smaller, localized changes to the surface. The equations create these visual effects; they are not a validated physical simulation.

Geometry and terrain

The field contains **1,040 pillars**, arranged in **52 columns × 20 rows**. Each pillar is a square column constructed via `THREE.BoxGeometry(1, 1, 1)` and rendered with instanced geometries, exposing a left face, a right face, and an accent cap.

Terrain is generated procedurally on the CPU and GPU instead of being loaded as a prebuilt mesh. A `mulberry32` pseudo-random generator supports variation. The summit selection chooses between one and four summits, using thresholds of `0.15`, `0.50` and `0.84`; the corresponding code appears in the appendix.

Peaks use a terraced height profile with a `3.2` step factor. The profile smooths the transition between terrace levels while keeping the peaks visibly stepped instead of turning them into continuous slopes.

Movement between sections

In the first homepage section, the landscape combines low wave motion with mountain-like peaks at a placement of `+35vh`.

The transition to the second section is specified as a Y-axis flip involving the camera and scene, with a shift to `-32vh`. The terrain appears to hang from the top of the viewport. In `#continuation`, it recedes to the upper right as a smaller lattice.

Waves and cursor response

Ambient motion and terrain clicks generate circular wave trains. The wave shader combines a cosine wave with a second harmonic to shape the crest, then applies an asymmetric envelope and distance attenuation. This is a **Stokes-style wave profile**; the excerpt does not show a complete Gerstner displacement model.

The radial attenuation is `1.0 / sqrt(1.0 + dist * 0.07)`. This gradually reduces the wave’s contribution with distance; it is not inverse-square decay.

For hover interaction, the cursor’s two-dimensional position is projected into world space using inverted camera projection matrices. A Gaussian distance weight controls nearby elevation lift, and `damp()` smooths its change over time.

Elevation and color

The shader blends the signal and accent colors according to `vHeight`:

HEIGHT RANGE	COLOR BEHAVIOR
<code>vHeight < 0.18</code>	Titan White base color
<code>0.18 ≤ vHeight ≤ 0.88</code>	Smooth transition toward the blue accent
<code>vHeight ≥ 0.88</code>	Full accent contribution before the brightness multiplier

The multiplier rises from `1.0` to `1.4` as the blend reaches blue. Its purpose is to keep the darker accent visible against the background. The summit therefore uses a brightened blue, not the unmodified CSS value `#002FA7`.

5. Responsive behavior and accessibility

Viewport behavior

WIDTH	LAYOUT AND SCROLLING	LANDSCAPE
Desktop Ultrawide, <code>≥ 1920px</code>	Centered <code>1800px</code> content container preserving <code>23 : 49 : 28</code> track balance; full-slide snap navigation	Three.js WebGL with <code>aspect > 1.82</code> horizontal-fit lock, preventing boundary cutoff on 21:9 & 32:9 displays
Desktop Standard, <code>> 960px</code>	Asymmetric three-column layout (<code>23 : 49 : 28</code>); full-slide snap navigation on Homepage & About	Three.js stage with cursor magnet and interactive ripple waves
Desktop Intermediate, <code>821px-960px</code>	Compact three-column layout with 20px gap; full-slide snap navigation	Three.js stage scaled to viewport bounds
Compact / Mobile, <code>≤ 820px</code>	Single-column document flow, native scrolling and touch-oriented padding	Static SVG poster <code>#terrain-poster</code> ; WebGL completely disabled

The compact presentation avoids ongoing WebGL rendering and the desktop terrain scripts, saving bandwidth and GPU battery on mobile devices.

Desktop slide navigation

A consolidated wheel controller (`assets/js/snap-scroll.js`) manages slide transitions for both the Homepage and About pages. It normalizes `deltaMode` by treating pixel deltas directly, multiplying line deltas by `32` , and multiplying page deltas by `window.innerHeight` .

The input threshold is `4px` , accepting gentle wheel notches while ignoring micro-jitters. The animation lock lasts `550ms` , preventing another navigation request while the current transition settles.

ArrowDown, ArrowUp, PageDown, PageUp and Spacebar provide accessible keyboard navigation. Focus guards ensure keyboard navigation does not intercept text inputs or buttons.

Reduced motion

When `prefers-reduced-motion: reduce` is active:

- Entrance curtain animation is completely bypassed.
- Full-page snap transitions switch from smooth scrolling to instant `auto` scrolling.
- Sinusoidal terrain drift is disabled (`config.drift = 0` , `uDrift = 0`).
- Shockwave ripples and pointer lift attraction are suppressed.
- The wireframe terrain remains visible as a calm, static geometric structure.

Layering and readability

The content is placed above the landscape in the stacking order:

LAYER	SELECTOR	Z - INDEX
Header and navigation	<code>.site-header</code>	<code>100</code>
Main content	<code>main</code> , <code>.page</code>	<code>2</code>
WebGL landscape	<code>#terrain-stage</code>	<code>1</code>
2D fallback	<code>#terrain</code>	<code>0</code>

`main` and `.page` use transparent backgrounds, allowing terrain to remain visible behind them. The stacking order puts content in front of the canvas, but does not by itself guarantee sufficient contrast for every text-and-terrain combination. Opacity values and animated backgrounds remain relevant to readability.

6. Loading and rendering lifecycle

The desktop entrance uses a curtain of raised pillars that descend into the landscape. Its start is tied to scene initialization, with a timeout intended to release the loading state if initialization does not complete normally.

First-visit sequence

1. The browser requests `index.html` .
2. A script in `<head>` adds `curtain-pending` , showing a dark screen and logo.

3. Preload hints make `three.min.js` and `terrain.js` discoverable early on desktop.
4. During DOM parsing, the 2D canvas `#terrain` remains hidden while `curtain-pending` is active.
5. The dynamically inserted scripts execute in dependency order: Three.js first, then the terrain script.
6. Successful `createCurtain()` initialization changes the state to `curtain-running`.
7. The curtain settles over `1,600ms`, after which `curtain-running` is removed and the entrance completes.

A `4,000ms` safety timeout replaces the earlier `650ms` timer. The shorter timer could expose the 2D canvas before the terrain scripts finished downloading, preventing the intended curtain entrance. Tying the entrance to initialization avoids relying on a fixed download window.

The timeout is intended to release a stalled loading state; it does not guarantee a curtain entrance on every connection. Handling of a late successful initialization after fallback has already started still needs confirmation.

Script delivery

Both scripts have preload hints with `media="(min-width: 821px)"`. Dynamic script elements set `.async = false` before they are appended, preserving the dependency order while allowing the browser to begin fetching both resources without waiting for the first script's `onload` callback.

Rendering fallback

There are three rendering paths:

- **Desktop WebGL:** the full terrain, elevation shader, waves and cursor response.
- **2D Canvas fallback:** the standalone `#terrain` renderer provides a procedural wireframe when the WebGL path fails. It does not depend on Three.js.
- **Compact SVG:** `#terrain-poster` provides a static representation at widths of `820px` and below.

Both injected scripts assign `onScriptError` as their error handler. The 2D path is intended to keep a landscape available after script or WebGL failure, provided its own code and assets are available. This is a rendering fallback, not an offline delivery mechanism.

7. Editorial approach and publishing

Writing about the work

Portfolio copy should name the systems and describe what happened. “UniFi routing, Proxmox virtualization, Docker containers, and automated storage” tells readers more than a general claim about infrastructure expertise.

Personal context belongs where it explains a decision:

The homelab started as a way out of more than \$200 a month in recurring subscriptions. Running services for other people turned routing, storage, and recovery into disciplined operational work.

Incident writing follows the same approach: describe the failure, the misleading evidence and the lesson. Article titles include:

- “The disk that read 82% free and was 99.9% full”
- “The host wasn’t dead and the backups weren’t real: both passed every check anyway”
- “I diagnosed the same outage twice and got two different answers”

Claims about scale, availability or performance need supporting evidence; when it is missing, describe the observation and its limits.

Blog generation

The blog uses a small dependency-free Node.js build pipeline. Markdown files in `content/blog/` are the source of truth; files under `blog/` are generated output.

The publishing flow is:

```
content/blog/*.md + templates/*.html → scripts/build-blog.js → blog/*/index.html
```

`scripts/blog-utils.js` owns frontmatter parsing, escaping, slug generation and reading-time calculation.

`scripts/markdown.js` renders the supported Markdown subset. The build script validates required metadata, dates, categories and unique slugs before it replaces the generated blog directory.

The generator produces standalone HTML articles, previous/next navigation and a filterable archive. It uses each post's frontmatter `summary` as its meta description and supports the categories **Homelab**, **Projects** and **Web**.

The homepage's selected-post title, date and reading time are synchronized from the same source metadata.

Articles are delivered as crawlable static HTML, with no client-side hydration framework or tracking scripts in the specified architecture.

8. Technical reference

Design tokens

Tokens defined in `assets/css/tokens.css` (and bundled into `assets/css/style.css`):


```

:root {
  /* Brand Color Palette */
  --color-black: #08090C;
  --color-white: #E5E7EB;
  --color-slate: #4B5565;
  --color-klein: #002FA7;

  /* Semantic tokens */
  --ink: var(--color-black);
  --structure: #0a0b0e;
  --signal: var(--color-white);

  /* Calibrated contrast tokens (tuned for WCAG AAA against --ink) */
  --signal-high: rgba(229, 231, 235, .82);
  --signal-48: #8892a0;
  --signal-24: rgba(75, 85, 101, .38);
  --signal-12: rgba(75, 85, 101, .18);
  --signal-10: rgba(75, 85, 101, .18);
  --accent: var(--color-klein);

  /* Typography primitives */
  --kyoto: "PP Kyoto", Georgia, serif;
  --authentic: "AUTHENTIC Sans", Arial, sans-serif;
  --pitch: "Pitch Sans", "Cascadia Mono", monospace;

  /* Responsive spatial tokens */
  --edge: clamp(20px, 2.15vw, 42px);
  --head-space: clamp(132px, 15vh, 176px);
  --page-x: clamp(18px, 3vw, 58px);
}

```

Components and implementation locations

The modular files under `assets/css/` are the CSS source. `scripts/build-css.js` concatenates them in a fixed order into `assets/css/style.css`; the bundle should be regenerated instead of edited independently.

IDENTIFIER OR PATH	RESPONSIBILITY
<code>.site-header</code>	Identity and primary navigation
<code>main</code> , <code>.page</code>	Content structure and desktop grid
<code>#continuation</code>	Final homepage section: About, selected writing and contact
<code>#terrain-stage</code>	Three.js rendering layer
<code>#terrain</code>	Standalone 2D Canvas fallback
<code>#terrain-poster</code>	Compact-view SVG illustration
<code>curtain-pending</code> , <code>curtain-running</code>	Entrance lifecycle classes
<code>createCurtain()</code>	Curtain initialization
<code>onScriptError</code>	Script failure handler
<code>assets/js/three.min.js</code>	Three.js dependency
<code>assets/js/terrain.js</code>	Terrain implementation
<code>assets/css/*.css</code>	Modular design tokens, layout, components and page styles
<code>assets/css/style.css</code>	Generated CSS bundle used by the pages
<code>templates/blog-post.html</code>	Template for an individual generated article
<code>templates/blog-index.html</code>	Template for the generated blog archive
<code>scripts/build-css.js</code>	CSS bundle generation
<code>scripts/blog-utils.js</code>	Shared frontmatter, slug, escaping and reading-time helpers
<code>scripts/markdown.js</code>	Renderer for the supported Markdown subset
<code>scripts/build-blog.js</code>	Blog validation and static route generation
<code>scripts/verify.js</code>	Source/output, link, syntax and build-consistency checks

9. Appendix: implementation excerpts

These excerpts show individual mechanisms, not complete standalone implementations. The identifiers and numeric values match the project specification.

A. Desktop grid

```
.page {
  display: grid;
  grid-template-columns:
    minmax(190px, 23fr)
    minmax(410px, 49fr)
    minmax(240px, 28fr);
  grid-template-rows: repeat(3, minmax(0, 1fr));
  gap: 18px clamp(24px, 2.5vw, 48px);
}
```

B. Summit selection and terracing

```
// Weighted selection of one to four summits.
const roll = random();
const count = roll < 0.15 ? 1 : roll < 0.50 ? 2 : roll < 0.84 ? 3 : 4;
```

The terrace profile first measures normalized elliptical distance from a summit:

```
r = sqrt(((u - summitX) / summitWidth)2 + ((d - ridgeCenter) / ridgeDepth)2)
raw = cos(r × π / 2)
t = 3.2 × raw
steep = clamp((frac(t) - 0.15) / 0.70, 0, 1)
height = (floor(t) + steep2 × (3 - 2 × steep)) / 3.2
```

The calculation runs only while `r < 1`. Here, `u` and `d` are normalized field coordinates, `frac` is the fractional part and `clamp` limits the value to the stated range. The result is multiplied by the summit's weight before contributing to the final pillar height.

C. Wave profile

```
// Cosine profile with a second harmonic to shape the crests.
const float k = 1.396; // Approximately 2 * PI / 4.5 wavelength.
float phase = -dr * k;
float stokes = (cos(phase) + 0.30 * cos(2.0 * phase) + 0.20) * 0.75;

// Narrow leading envelope and longer trailing envelope.
float envelope = (dr > 0.0)
  ? exp(-pow(dr / 1.5, 2.0))
  : exp(-pow(dr / 14.0, 2.0));

// Distance attenuation.
float radialDecay = 1.0 / sqrt(1.0 + dist * 0.07);
height += stokes * envelope * radialDecay * uShockStrength * uShock[i].w;
```

D. Elevation color

```
// Blend from signal to accent as height increases.
float ikbProgress = smoothstep(0.18, 0.88, vHeight);

// Increase brightness toward the summit.
float luminanceBoost = 1.0 + ikbProgress * 0.40;
vec3 finalColor =
    mix(uSignalColor, uAccentColor, ikbProgress) * luminanceBoost;
```

E. Cursor influence

```
W_i = exp(-(d_i / influencePixels)2 × 3.4)
```

The weight decreases with distance from the projected cursor. `damp()` smooths the resulting elevation response.

F. Wheel navigation

```
window.addEventListener('wheel', (e) => {
  if (window.innerWidth <= 820) return;
  // deltaMode: 0 = pixels, 1 = lines, 2 = pages.
  const delta = e.deltaMode === 1
    ? e.deltaY * 32
    : e.deltaMode === 2
    ? e.deltaY * window.innerHeight
    : e.deltaY;

  if (Math.abs(delta) < 4) return;
  if (isAnimating) {
    e.preventDefault();
    return;
  }

  if (delta > 0 && currentPageIndex < pages.length - 1) {
    e.preventDefault();
    goToPage(currentPageIndex + 1);
  } else if (delta < 0 && currentPageIndex > 0) {
    e.preventDefault();
    goToPage(currentPageIndex - 1);
  }
}, { passive: false });
```

This excerpt covers wheel input. The keyboard bindings and focus guards described earlier are not shown in it.

G. Script preloading and insertion

```
<link rel="preload"
      href="/assets/js/three.min.js"
      as="script"
      media="(min-width: 821px)">
<link rel="preload"
      href="/assets/js/terrain.js"
      as="script"
      media="(min-width: 821px)">
```

```
// Allow fetching without an onload chain; retain dependency order.
const s1 = document.createElement('script');
s1.src = "/assets/js/three.min.js";
s1.async = false;
s1.onerror = onScriptError;

const s2 = document.createElement('script');
s2.src = "/assets/js/terrain.js";
s2.async = false;
s2.onerror = onScriptError;

document.head.append(s1, s2);
```